Classic planning with PDDL

Aleksandra Słomska
Zofia Narloch

March 4, 2026

1 Exercise 1.1: Emergency Service Logistics, Initial Release

In the first exercise, a domain and two small problems were created. To present the outcome of each problem, plan files have been attached. In both problems people are in the same room.

```
(define (problem problem1) (:domain p1)
(:objects
    drone1 - drone
    crate1 - crate
    person1 - person
    room1 depot - location
    food medicine - content
    left right - arm
)
(:init
    (person-at person1 room1)
    (crate-at crate1 depot)
    (content-crate food crate1)
    (drone-at drone1 depot)
    (empty-arm drone1 left)
    (empty-arm drone1 right)
)

(:goal (and
   (has-content-person person1 food)
))

)
```

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
```

Listing 1: First problem and the outcome - one person and one box

```
(define (problem problem2) (:domain p1)
(:objects
    drone1 - drone
    crate1 - crate
    crate2 - crate
    crate3 - crate
    person1 - person
    person2 - person
    room1 depot - location
    food medicine - content
    left right - arm
)

(:init
    (person-at person1 room1)
    (person-at person2 room1)
    (crate-at crate1 depot)
    (crate-at crate2 depot)
    (crate-at crate3 depot)
    (content-crate food crate1)
    (content-crate medicine crate2)
    (content-crate food crate3)
    (drone-at drone1 depot)
    (empty-arm drone1 left)
    (empty-arm drone1 right)
)

(:goal (and
   (has-content-person person1 food)
   (has-content-person person2 medicine)
))

)
```

```
( PICK-UP-CRATE CRATE1 DEPOT DRONE1 RIGHT )
( PICK-UP-CRATE CRATE2 DEPOT DRONE1 LEFT )
( DRONE-MOVE DEPOT ROOM1 DRONE1 )
( DROP-CRATE ROOM1 PERSON1 CRATE1 RIGHT FOOD DRONE1 )
( DROP-CRATE ROOM1 PERSON2 CRATE2 LEFT MEDICINE DRONE1 )
```

Listing 2: Outcome of the second problem - two people and three boxes

2 Exercise 1.2: Problem Builder in Python

In the second exercise, the performance of the FF planner was tested. We created a Python script that runs the generate-problems.py algorithm, executing the generated problems until the execution time exceeded one minute. The problem size was incremented by two after each execution. Maximum size within one minute reached 45.

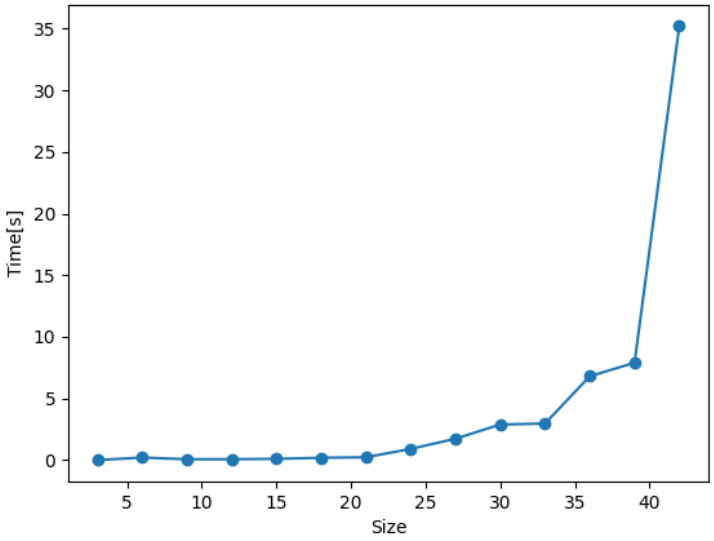

Figure 1: Size of the problem and the time required to find a solution in the tests

3 Exercise 1.3: Performance Comparison of Search Algorithms and Heuristics

3.1 First Part

Algorithm	Heuristic	Largest Solved Size	Time (s)	Plan Length	Optimal
BFS	None	7	24.59	22	Yes
IDS	None	3	0.71	10	Yes
A*	hMAX	5	9.95	17	Yes
GBFS	hMAX	7	10.94	26	No

Table 1: Performance Comparison of Search Algorithms and Heuristics (Exercise 1.3.1)

The results of the performance of every algorithm show that only GBFS with heuristic hMAX is not optimal. The reason for that is the longest plan length. This is because GBFS relies on the heuristic to rush toward the goal quickly rather than exhaustively searching for the perfect path. The same size of the problem was solved by the BFS algorithm without any heuristics. However, it took more time, because it blindly checks every possible combination of moves. On the other hand, the A* algorithm only reached size 5 because the calculation of the hMAX heuristic for every node caused it to time out. Algorithm IDS solved the smallest problem of them all. IDS intentionally wipes its memory and starts over every time it increases its search depth, so it concentrates on memory efficiency.

In this part every action in domain file has the same cost, that is why algorithms like BFS guarantee the shortest path by expanding all nodes at the current search depth before moving deeper. Secondly, at every step, the drone has many valid moves, which creates many possibilities, which can cause problems for some algorithms like BFS.

3.2 Second Part

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
GBFS	NONE	0.37	26
GBFS	hMAX	10.68	26
GBFS	hFF	0.28	26
GBFS	hADD	0.35	27
GBFS	LANDMARK	0.29	28
EHC	NONE	0.31	28
EHC	hMAX	Timeout	$>$ 60s
EHC	hFF	0.39	28
EHC	hADD	0.44	28
EHC	LANDMARK	0.27	31

Table 2: Performance Comparison of GBFS and EHC with Various Heuristics (Exercise 1.3.2)

Based on the results from Exercise 1.3.1, the largest problem that Greedy Best-First Search (GBFS) could handle within the 1-minute time limit was size 7. This problem size was generated and tested for the algorithms GBFS and EHC with heuristics hMAX, hFF, hADD and Landmark. Only algorithm EHC with heuristic hMAX exceeded the time limit. The GBFS solved a size 7 problem with hMAX in under 11 seconds.

EHC is more memory-efficient than GBFS, since it discards alternative paths once it makes a choice. It causes the algorithm to be fast and use minimal memory. However, it causes problems when the chosen path reaches dead-end, since it can't go back. The GBFS keeps all values in memory, so it can return if the path is unpromising, but at cost of taking much more memory.

The FF planer intelligently combines the strengths of both algorithms to maximize speed and reliability. The FF planner primarily relies on EHC paired with the hFF heuristic. However, if EHC gets trapped in a dead-end, the planner automatically changes to GBFS. The result of that is the planner can backtrack out of the dead-end, guaranteeing that a solution will eventually be found.

3.3 Third Part

Algorithm	Heuristic	Time (s)	Plan Length
A*	NONE	6.51	17
BFS	NONE	0.77	17
IDS	NONE	Timeout	> 60s
A*	hMAX	9.18	17
A*	LMCUT	47.76	17

Table 3: Performance of Optimal Planners and Admissible Heuristics (Exercise 1.3.3)

An admissible heuristic is one that never overestimates the true cost to reach the goal, guaranteeing that an algorithm like A* will find an optimal solution. Among the heuristics implemented in Pyperplan, hMAX and LMCUT (Landmark-Cut) are admissible.

The results show that almost every algorithm has the plan length equal to 17. The best-performing algorithm was BFS without any heuristics. The reason for that is the same cost for every action. Moreover, adding the heuristics to algorithm A* resulted in exceeding the time about 1.4 times for hMAX, and over 7 times for LMCUT. It is because of the necesity to calculate these complex heuristics at every node.

Finally, IDS timed out because a plan length of 17 steps requires it to wipe its memory and completely regenerate the search tree from scratch 17 times, which causes to exceed the 1-minute limit.

4 Exercise 2.1: Problem Builder in Python

```
(define (problem drone_problem_d1_r1_l2_p2_c2_g2_ct2)
(:domain p2)
(:objects
  drone1 - drone
  depot - location
  loc1 - location
  loc2 - location
  crate1 - crate
  crate2 - crate
  food - content
  medicine - content
  person1 - person
```

```
  person2 - person
  carrier1 - carrier
  N0 - num
  N1 - num
  N2 - num
  N3 - num
  N4 - num
  N5 - num
  N6 - num
  N7 - num
)
(:init
  (drone-at drone1 depot)
  (crate-at crate1 depot)
  (crate-at crate2 depot)
  (carrier-at carrier1 depot)
  (carrier-capacity carrier1 N4)
  (crates-in-carrier carrier1 N0)
  (content-crate food crate1)
  (content-crate medicine crate2)
  (person-at person1 loc1)
  (person-at person2 loc2)
  (next-num N0 N1)
  (next-num N1 N2)
  (next-num N2 N3)
  (next-num N3 N4)
  (next-num N4 N5)
  (next-num N5 N6)
  (next-num N6 N7)
)
(:goal (and
  (drone-at drone1 depot)
  (has-content-person person2 food)
  (has-content-person person2 medicine)
))
)
```

```
( PUT-CRATE-IN-CARRIER CRATE1 DEPOT DRONE1 CARRIER1 N0 N1 )
( PUT-CRATE-IN-CARRIER CRATE2 DEPOT DRONE1 CARRIER1 N1 N2 )
( MOVE-CARRIER DEPOT LOC2 DRONE1 CARRIER1 )
( PICK-AND-DROP CRATE1 LOC2 DRONE1 CARRIER1 PERSON2 FOOD N2 N1 )
( PICK-AND-DROP CRATE2 LOC2 DRONE1 CARRIER1 PERSON2 MEDICINE N1 N0
   )
( MOVE-CARRIER LOC2 DEPOT DRONE1 CARRIER1 )
```

Listing 3: Outcome of the second problem - two people and three boxes

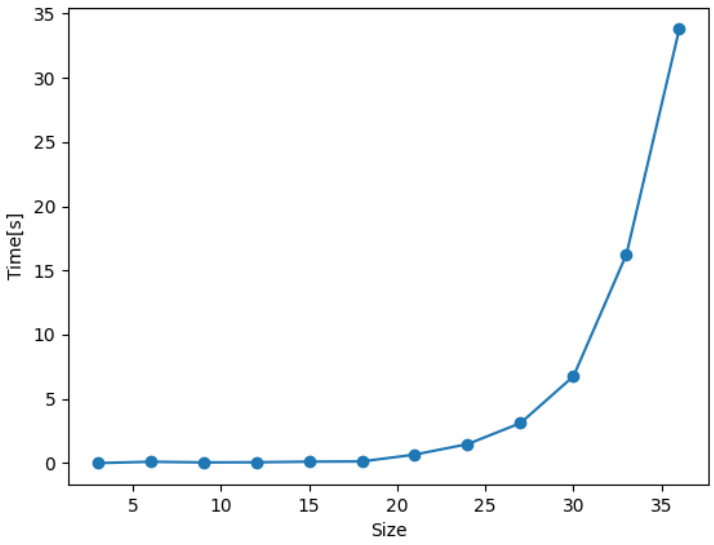

Figure 2: Size of the problem and the time required to find a solution in the tests

Algorithm	**Heuristic**	**Time (s)**	**Plan Length**
GBFS	NONE	0.27	24
GBFS	hMAX	Timeout	$>$ 60s
GBFS	hFF	0.30	24
GBFS	hADD	0.29	27
GBFS	LANDMARK	0.20	28
EHC	NONE	0.20	27
EHC	hMAX	Timeout	$>$ 60s
EHC	hFF	0.21	27
EHC	hADD	0.19	24
EHC	LANDMARK	0.44	35

Table 4: Performance Comparison of GBFS and EHC with Various Heuristics (Exercise 2.1)

Algorithm	Heuristic	Time (s)	Plan Length
A*	NONE	2.57	16
BFS	NONE	0.82	16
IDS	NONE	Timeout	> 60s
A*	hMAX	6.60	16
A*	LMCUT	17.91	16

Table 5: Performance of Optimal Planners and Admissible Heuristics (Exercise 2.1)

5 Exercise 2.2: Problem Builder in Python

5.1 First Part - satisficing planners

Test run with lama-first alias

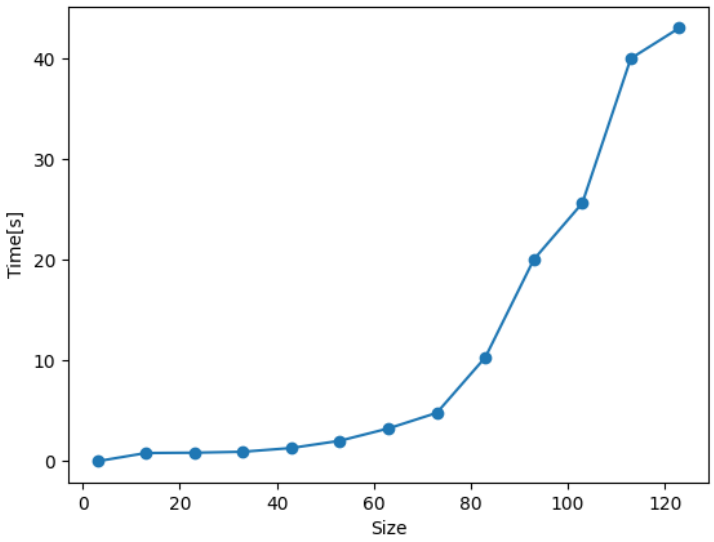

Figure 3: Size of the problem and the time required to find a solution in the tests

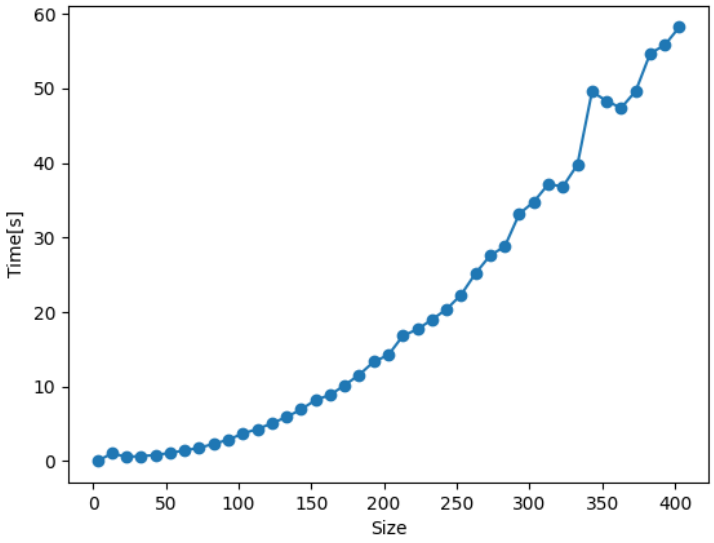

Figure 4: Size of the problem and the time required to find a solution in the tests with seq-sat-fdss-2 alias

Test run with lama-first alias

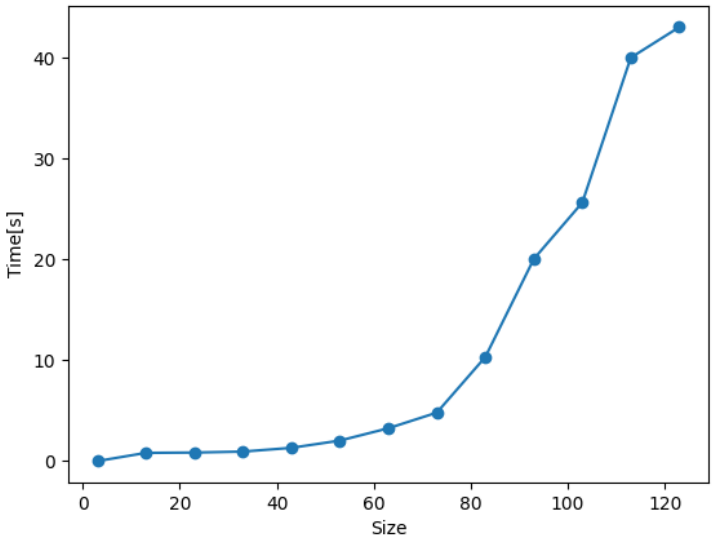

Figure 5: Size of the problem and the time required to find a solution in the tests

Planner	Alias	Max problem size	Max size cost	Cost(size 3)
Metric-ff	NONE	10 (time 0.1s)	2144	834
Fast Downward	lama-first	120	32840	834
Fast Downward	seq-sat-fdss-2	400	??	0
Fast Downward	seq-sat-fd-autotune-2	6	834	518

Lama-first and seq-sat-fdss-2 biggest problem sizes are esitmates, sizes were being incremented by 10. Metric-ff gets overflown when size of the problem exceeds 10.

5.2 Second Part - optimal planners

Planner	Alias	Max problem size	Max size cost	Cost(size 3)
Fast Downward	seq-opt-lmcut	1023	32840	518
Fast Downward	seq-opt-fdss-2	400	??	0
Fast Downward	seq-opt-bjolp	6	1023	518